

MANUAL DE USUARIO

ELABORADO POR:

DÍAZ RUIZ ISRAEL, IGNACIO CORTÉS ATZIN MAXELA, RÍOS RIVERA FERNANDA ANAHÍ, SANDOVAL CALDERÓN GERARDO

VERSIÓN: 1.0 (PROTOTIPO)

INTRODUCCIÓN AL SISTEMA

El proyecto propone el desarrollo e implementación de un Sistema de Monitoreo y Alerta Temprana de Contaminación Atmosférica centrado en la detección y cuantificación de emisiones de tabaco. Este sistema integra un enfoque con tecnologías avanzadas como sensores ambientales, cámaras termográficas y algoritmos de visión por computadora. Los datos recolectados se procesan mediante ciencia de datos y aprendizaje automático para modelar la relación entre la actividad de fumar y los incrementos en partículas contaminantes, generando así niveles de alerta temprana para usuarios y transeúntes. Esto es importante porque aborda la problemática de contaminación atmosférica, haciendo especial énfasis en el consumo de tabaco como una fuente significativa, concentrada y persistente de dichas emisiones.

ESPECIFICACIONES DEL SISTEMA

Antes de usar y hacer funcionar el prototipo, es importante destacar las herramientas que se tuvieron para el buen funcionamiento del proyecto, se optó por el uso de dos máquinas virtuales con las siguientes especificaciones mínimas:

Sistema Operativo	Tecnología de Virtualización	CPU	Memoria RAM	Espacio en disco
Ubuntu	Proxmox	8 núcleos (Xeon)	16 GB	200 GB
Windows 11	VMware	4 núcleos	16 GB	200 GB

La base de datos se guarda en la plataforma de Supabase, que está basada en PostgreSQL.

GUÍA DE OPERACIÓN (PASO A PASO)

MÁQUINA VIRTUAL A (UBUNTU)

Comunicación del Sensor de partículas (Davis)

Para poder usar el sensor Davis primero, debe tener la aplicación Davis Airlink para conectar al internet, este sensor tiene su propio manual de usuario, favor de consultarlo para errores o consultas del propio sensor:

https://cdn.shopify.com/s/files/1/0515/5992/3873/files/07395_384_Manual_AirLink_UG_RevC_web.pdf?v=1656098534

Para efectos prácticos, una vez que ya tenga el sensor conectado a la red wifi, primeramente, debe crear en la máquina virtual (VM) un entorno para Python (regularmente al mismo nivel de carpeta que los archivos que se van a ocupar), una vez creado deberá correr esta línea en la terminal con enfoque a la ruta especificada para la librería que va a usar:

```
pip install httpx python-dotenv
```



A la vez en la misma carpeta deberá crear un archivo para las variables de entorno (.env) y completar los siguientes campos con sus respectivas credenciales.

DAVIS_API_KEY

DAVIS_API_SECRET

DAVIS_STATION_ID

SUPABASE_URL

SUPABASE_KEY

Las primeras tres claves son encontradas en el usuario del Airlink y las últimas dos son la base de datos en donde se guardarán.

Una vez preparado lo anterior deberá ir a los archivos ejecutables del disco y en la carpeta que dice base de datos encontrará el código (Sincronizacion_supabase.py), deberá descargarlo y guardarlo en la carpeta donde esté su entorno de Python; finalmente deberá configurar Cron para que este código se ocupe cada 5 min, para esto deberá abrir una terminal y poner:

crontab -e

Al final de todo el documento que abrió deberá escribir:

```
*/5 * * * * /ruta/a/tu/venv/bin/python /ruta/a/tu/script/sync_supabase.py >>
/ruta/a/tu/script/cron_supabase.log 2>&1
```

Este es un ejemplo de cómo se debe poner, pero debe asegurarse de cambiar las rutas del entorno de Python y del código para Supabase.

Comunicación de la Cámara y su uso en YOLO

Para poder utilizar la cámara termográfica es necesario indicarle a la máquina virtual el puerto físico que este ocupa para poder acceder a las imágenes térmicas.

Una vez teniendo eso listo, abre la terminal y comprueba que realmente haya identificado la cámara.

Si aún no se ha hecho, instala las dependencias necesarias en un entorno virtual de Python3 (para evitar instalarlas de forma global y trabajar en un entorno controlado) y corre el archivo *cam_isla_pagina.py* que contiene el código necesario para abrir el servidor de flask y correr el modelo de YOLO para enviar tanto el video con las inferencias del modelo como los metadatos de las detecciones que haga. Para ello, ve hacia el directorio donde tengas tanto el entorno virtual como el archivo con el modelo y corre en la terminal:

source <env name>/bin/activate

python cam_isla.py

Reemplazando <env name> por el nombre que le hayas dado al entorno virtual. Esto abre el servidor de forma local para poder monitorear el video en vivo junto al JSON de metadatos que estará enviando al dashboard.



Este archivo nos indica si tiene acceso tanto a la cámara como al backend del dashboard para poder hacer tanto las inferencias como el envío de metadatos.

MÁQUINA VIRTUAL B (WINDOWS 11)

Como datos iniciales en esta VM el dashboard se encuentra alojado en dos partes que son: Backend y Frontend. Antes de adentrarnos en cada una, primero se debe instalar lo primordial como es Python.

Se debe instalar Python 3.11 o superior.

La versión recomendada es 3.11.x LTS y puede ser descargada en:

<https://www.python.org/downloads/windows/>

Una vez teniendo el ejecutable (.exe) es importante poner la casilla con un check en "Add Python to Path" durante la instalación y verificar si la instalación se hizo correctamente.

Otra herramienta que será importante Node.js 16+ (con npm)

Versión recomendada: Node.js 18.x LTS

Descarga: nodejs.org

Instalación:

- A. Descargar instalador .msi para Windows x64
- B. Ejecutar como administrador
- C. Seleccionar "Automatically install necessary tools for native modules"
- D. Completar instalación

Una vez descargados estos dos recursos, volvemos a la carpeta de ejecutables, del disco, y en la parte de dashboard se encontrarán todos los archivos necesarios para correr todo el prototipo.

DASHBOARD (BACKEND)

El principal elemento que se necesita son las variables de entorno (.env), este debe estar a la altura de la carpeta de backend y debe contener los siguientes valores.

SUPABASE_URL

SUPABASE_KEY

Puerto del servidor (por defecto)

PORT=3001

URL del API Davis WeatherLink (si aplica)

DAVIS_API_KEY

DAVIS_API_URL

Configuración de datos estáticos

DATA_DIR=./static_data



Al tener este archivo listo, se debe abrir una terminal (preferentemente que no sea powershell) en la ruta de la carpeta de backend y se debe ejecutar:

```
npm install
```

Este comando nos permite instalar las dependencias y para verificar opcionalmente puede correr:

```
npm list
```

Teniendo esto listo se corre:

```
npm run dev
```

Para ejecutar el backend

Ahora se debe crear un entorno para python con el siguiente comando:

```
python -m venv venv
```

Y el comando para iniciarlo:

```
.\venv\Scripts\Activate.ps1
```

Cuando finalice se deben descargar las librerías necesarias dentro del entorno:

```
pip install -r requirements.txt
```

Y verificas con:

```
pip list
```

En una terminal aparte deberás colocarte otra vez en la carpeta backend, correr el entorno de Python y correr el código `api_modelo.py`, esto para activar el modelo de predicción para los datos del sensor de partículas.

Es totalmente normal que en las primeras horas el código este mandando un get 404 ya que debe recolectar datos para realizar la predicción.

Finalmente, en una terminal aparte deberá correr esta línea en una terminal de powershell como administrador:

```
ssh -R 3001:localhost:3001 -L 5000:localhost:5000 [Nombre de la Maquina A]@[IP]
```

Con esta línea puedes conectar la cámara termográfica que está conectada a la VM A con el backend de la VM B.

DASHBOARD (FRONTEND)

Para el frontend se debe abrir una terminal dedicada (preferentemente que no sea powershell) y se corre el siguiente comando:



```
npm install
```

seguido de:

```
npm run dev
```

Para abrir el dashboard basta con hacer CTRL + click sobre el enlace mostrado que es nuestro host local, lo que abrirá el enlace en el navegador por defecto que tengas.

Para abrirlo en el navegador de tu preferencia, basta con copiar el enlace mostrado en la terminal y pegarlo en la barra de búsqueda superior para acceder.

POSIBLES ERRORES

PYTHON NO DEJA INSTALAR ULTRALITICS

Este problema se ocasiona ya que no se tiene pre-instalado las herramientas de cuda. No es necesario usar el controlador de la gráfica dedicada, ya que el modelo puede correr usando los núcleos del procesador (aunque no es recomendado si cuenta con una tarjeta gráfica de Nvidia).

Soluciones:

Desde la página de Nvidia:

<https://developer.nvidia.com/cuda/toolkit>

Te pedirá que te registres y deberás escoger tu sistema operativo, arquitectura e instalador que quieras descargar, para nuestro caso es el ejecutable (.exe). Una vez que lo ejecute en la parte de drivers, desactívalo en caso de usar solo el puro el procesador, en caso de que lo requiera actívalo.

NPM: COMANDO NO ENCONTRADO

Error: npm: The term 'npm' is not recognized

Soluciones:

1. Verificar la instalación de Node.js:

```
node --version
```



```
npm --version
```
2. Si no aparece, reinstalar Node.js:
 - Desinstalar desde "Control Panel" → "Programs and Features".
 - Descargar Node.js 18 LTS nuevamente.
 - Ejecutar como administrador.
 - Seleccionar instalar npm y herramientas de compilación.
 - Completar instalación.
3. Cerrar y reabrir la terminal donde estaba corriendo.



ERROR EN NPM INSTALL

Error: npm ERR! Could not resolve dependency... o npm notice

Soluciones:

1. Limpiar caché de npm:
`npm cache clean --force`
2. Eliminar node_modules y package-lock.json:
`Remove-Item -Recurse -Force node_modules`
`Remove-Item package-lock.json`
3. Reinstalar:
`npm install`

Si el error persiste:

1. Verificar versión de npm:
`npm --version`
2. Actualizar npm a última versión:
`npm install -g npm@latest`
3. Reinstalar:
`npm install`

ERROR DE COMPILACIÓN DE MÓDULOS NATIVOS EN PYTHON

Error: Microsoft Visual C++ 14.0 is required...

Building wheel for xgboost failed

Soluciones:

1. Instalar Visual Studio Build Tools:
 - Descargar desde: <https://visualstudio.microsoft.com/downloads/>
 - Ejecutar instalador.
 - Seleccionar "Desktop development with C++".
 - Completar instalación.

Alternativamente, usar Wheel precompilado:

`pip install xgboost --only-binary :all:`

PUERTO 3001 YA ESTA EN USO

Error Port 3001 is already in use o EADDRINUSE: address already in use :::3001



Identificar proceso usando el puerto:

```
netstat -ano | findstr ":3001"
```

Salida esperada:

```
TCP    0.0.0.0:3001    0.0.0.0:0      LISTENING      12345
```

Soluciones:

- Terminar el proceso (modo seguro):
Cambiar el número 12345 por el PID de la salida anterior

```
Stop-Process -Id 12345 -Force
```

- Usar un puerto diferente:
PORT=3002

- Y modificar BACKEND/server.js (línea con listen):

```
const PORT = process.env.PORT || 3002;
```

```
app.listen(PORT, () => console.log(`Servidor en puerto ${PORT}`));
```

O reiniciar la computadora como último recurso.

FRONTEND NO SE CONECTA CON EL BACKEND

Error en consola del navegador: Failed to fetch from http://localhost:3001

- Verificar que el backend este corriendo:

```
netstat -ano | findstr ":3001"
```

Debe haber una línea con LISTENING

- Intentar acceder directamente: http://localhost:3001

Debe recibir respuesta JSON (no error 404)

- Verificar CORS en BACKEND/server.js:

Debe existir:

```
app.use(cors({
  origin: '*', // 0 especificar http://localhost:5173
  credentials: true
}));
```

- Reiniciar Backend:

- Cerrar terminal del Backend.



- Esperar 2 segundos.
- Ejecutar `npm run dev` nuevamente.
 - a. No hay datos en el dashboard.
- a) Verificar que el Backend Python está corriendo:


```
netstat -ano | findstr ":3000"
```
- b) Verificar credenciales de Supabase:
 - Abrir `BACKEND/.env`
 - Verificar que `SUPABASE_URL` y `SUPABASE_KEY` están correctos.
 - Si están vacíos, completar con valores reales.
- c) Verificar conectividad a Supabase:

En `BACKEND`, activar `venv` y ejecutar:

```
python -c "import requests; r = requests.get('https://tu-proyecto.supabase.co');
print(r.status_code)"
```

Debe mostrar código de estado (200, 301, etc., no error)

- d) Revisar logs del Backend:
 - Mirar la terminal del Backend.
 - Buscar mensajes de error o "Connection refused".
 - Si hay JSON parse errors, verificar formato de datos.

CÁMARA NO SE RECONOCE EN `CAM_ISLA.PY`

Error: `IndexError: list index out of range` al intentar acceder a cámara

- a) Verificar cámara conectada:

Abrir Configuración de Cámara:

```
Start-Process ms-settings:privacy-webcam
```

O usar software como "Camera" (Cheese para linux)

- b) Cambiar índice de cámara:

En `BACKEND/cam_isla.py` o donde se use cámara:

Probar diferentes índices

```
CAMERA_INDEX = 0 # Por defecto
```

Si no funciona, intentar:

```
CAMERA_INDEX = 1
```

```
CAMERA_INDEX = 2
```

- c) Reinstalar drivers de cámara:
 - Ir a "Device Manager" (Administrador de dispositivos).



- Expandir "Cameras".
- Clic derecho → "Update driver".
- Seleccionar "Search automatically for drivers".

